The Distributed Run Manager
A C++ Library for Distributed Processing

Brian P. Harrawood*

Abstract

*The **DistributedRunManager** (DRM) is a tool for distributing a computing task among a number of computer systems for parallel processing on a network. It is a small collection of C++ classes contained in the **catlib** library originally designed to accelerate synthetic image production for medical CT images. The tool is, however, a general solution for distributing a software task across diverse, multiple hardware platforms. Employing asynchronous communications, extensive threading and a simple socket protocol, the DRM offers the application developer a flexible and fault-tolerant tool to accelerate their software.*

The model for distribution is an unbounded client-server in which the clients perform the computing tasks and the server maintains the state of the overall progress. The tool is tolerant of individual processes (clients) joining and/or exiting the work pool before the entire run is complete providing dynamic load balancing.

*The DRM self-configures at runtime using a novel C++ class, the **ElectionManager**. This class cooperatively selects a server from among the processes which were started and ensures agreement going forward on which process in the flight acts as the server.*

Introduction

The advances in High Performance Computing (HPC) have been greatly accelerated in recent years specifically for computationally intensive tasks such as scientific modeling and machine learning. This has led to the development of more affordable hardware architectures which perform massively parallel computations. However, a parallel programming model is required for application software to make use of this massive parallelism.

Currently, the Message Passing Interface (MPI) standard (see `https://docs.open-mpi.org/`) is used for building distribution models but requires considerable design and coding to integrate with an application [1]. The DRM library offers a clear, concise API to the application developer relieving them of the coding of network communications, distribution architecture, fault tolerance and restart capability.

The `g4DistributedRunManager` described in [2] is an early precursor to the DRM. The current DRM decouples the code from the **GEANT4 Toolkit** [3] and adds a new class, the `ElectionManager`. This class permits DRM enabled applications to self-organize at runtime and select a process from among themselves to act as a server to manage the state of the overall task.

The DRM is open source released under GPLv2 and is available on GitHub. User documentation and sample code are provided there. This document is not intended as a user guide, but rather an in-depth examination of the design of the library.

*Corresponding author: brian.p.harrawood@gmail.com

A quick note on the nomenclature used. The word *run* is used in the sense of a "production run" to indicate an entire task. For instance, it might refer to performing a simulation or an estimation. The DRM manages a run comprised of multiple *processes* (i.e., Linux processes). The DRM harnesses multiple processes and uses them to perform a run.

Design Overview

A distinction is often made between *parallel processing* and *distributed processing* [4]. This difference regularly hinges on whether the run is accomplished using a single node (i.e., parallel) or multiple nodes connected via a network (i.e., distributed). The DRM is designed to use a blend of both approaches, employing multiple processes on a single node and/or multiple nodes on a local network.

The DRM scheme for maintaining the state of the run is the client-server model. In particular, it is the Single Server Multiple Client System (SSMCS) [5] in which one server provides individual parts of the work to multiple clients in the form of a "ticket". The system is unbounded in the number of clients it can support, and clients are free to come and go as the run evolves.

The server process keeps track of which tickets are completed, which are running, and which are ready-to-run. Using message passing, a client process requests a piece of the run (i.e., a ticket) from the server. It completes that portion of the run and then reports back to the server. A DRM application will also run as a stand-alone process in which the server asks itself for tickets and processes them sequentially.

The DRM "server" should not be confused with the term server used for a persistent process such as the `httpd` server. The DRM server is created at the beginning of a run and exits when the run is complete. There are no daemon processes used.

The network bandwidth requirements for communications between clients and server are modest. Each client must have local access to any and all the data needed to do any piece of the task. The communications between server and client are reduced to just *which* task to do next and can usually be represented as an integer or short string.

Designating the server - Election Management

The client-server algorithm requires that there is one and only one server. To automate this, the DRM uses the `ElectionManager` class which chooses a server from the running processes. This runtime election removes the need for a mechanism to do *a priori* designation of clients and server.

The distinction in the DRM between client and server processes is handled at runtime with logical operators. The library's single code base executes in either mode. The DRM process starts out as neither client nor server. The `ElectionManager` assigns one role or the other to the process and that role is then immutable.

When a process starts, it broadcasts to the network, seeking a server process. If a server answers, the process connects as a client. If no server answers and no other broadcasts are detected, the process becomes the server.

But, if there are other processes searching for a server and no server is responding, an election is held among the candidates to select one. (All processes in the *flight* are eligible candidates.) The cooperatively selected process is "elevated" to server and the remaining processes attach to it as clients.

Task Granularity - Tickets

The pieces of the run which are distributed to the clients are encoded in the C++ class, the `Ticket`. A run is divided into pieces which can be uniquely mapped to tickets. The amount of work included in a ticket is sometimes referred to as the granularity of the distribution. Each ticket completely describes a piece of the run to be executed within a single client process. If the overall run is imagined as a logical loop, then each ticket is an iteration of that loop.

Generally, each ticket uses different input data to generate its own output data: this is loosely a variation of the Single Instruction Multiple Data (SIMD) processing algorithm. The output data are written to file by the client process. At the end of the run the DRM benefits from a shared file system (e.g., NFS) to collate these files.

The tickets are dynamically distributed to clients. As a client comes free, it will be assigned the next ticket by the server. This allows DRM-enabled applications the flexibility to run with any number of clients. Additionally, the number of clients may change *during* the run and the DRM will gracefully accommodate.

Extensibility Considerations

The DRM is divided into classes which are opaque and classes which "hook" into the application. The opaque classes (`DistributedRunManager` and `ElectionManager`) offer little configuration while the other classes (`TicketManager` and `Ticket`) are abstract base classes which require application level coding.

Both the `DistributedRunManager` and the `ElectionManager` contain messaging details (e.g.; protocols, socket handling) and run logic (e.g.; client and server mode selection) which have been abstracted from the application programmer. It is anticipated that these details can be safely forgotten when designing a new application.

However, the `TicketManager` and `Ticket` need flexibility to accommodate the specifics of a distribution. The details of dividing a task into pieces are handled in these classes. Making them base classes provides the application programmer the flexibility to create their own newly derived classes to meet the specific needs of their application.

Design Constraints

The DRM is not designed as a general purpose problem solving tool. It is designed with particular types of problems, software environments, and hardware in mind. It is important to understand these constraints before applying it to your problem.

Problem Characteristics The DRM is only useful for solving a particular class of problems. The problems must be "parallel-izeable" in nature. In other words, they must require many repetitive iterations, each of which is independent of the any other. Iteration $i + n$ may not rely on output from iteration i.

The number of tickets must be calculable (there is currently no provision for iterating to a limit) and must be known at the start of the run. The number of tickets is also a factor in the overall throughput. This has to do with the granularity of the distribution.

A granularity which is too fine (too many, small pieces) may lead to ticket execution times comparable

to the transmission time of a ticket. A granularity which is too coarse (too few, large pieces) limits the expanse of the distribution and negates performance gains.

In practice, the DRM ticket data size is expected to be very small, e.g., just an integer or short string. The time to pack/unpack and transmit a ticket needs to be much, much less than the time to execute the ticket or the overall time spent on a run will be mostly spent passing messages.

Software Environment The DRM is an application level tool. It has no facility for starting, stopping, or reporting on system processes. To effectively use a DRM-enabled application, these "meta" activities must be managed by another agent: either manually at the command-line, or scripting methods or an executive manager such as slurm (see `https://slurm.schedmd.com`).

Although DRM applications run optimally on a managed HPC cluster, they are still useful without one. On a single multi-processor-node, multiple instances of the application can be run in succession from a command line. These instances then run in parallel to improve throughput.

Or, still just using the command line, multiple instances may be run on multiple nodes as a Beowulf cluster [6]. The Beowulf cluster technique requires network communication between nodes, shared user permissions, and a shared file system. These are all standard on a typical Linux network which has home or data directories shared.

In this environment, a user starts the application a number of times on one node, then logs onto another node and starts more there. The applications communicate between nodes with no further intervention. In this situation, the management of resources (CPU cores or GPU's) is left to the user.

A managed cluster has the user permissions, local network and networked storage systems already in place. A single DRM-enabled application can be submitted as an *array-job* with a cluster job scheduler. The cluster management software handles all resource allocation and the DRM handles the client-server designation and work flow among the processes.

Hardware There is no need for cooperating DRM nodes to be homogeneous. As long as the application can be built and run on a platform, it is eligible for inclusion. There is nothing about the design which requires a particular platform: e.g., big or little endianess, or operating system dependency.

The network offers more subtle constraints. Nodes must be reachable "locally"; i.e., on the same subnet. Firewall security may need to be relaxed. The `ElectionManager` does a limited broadcast to only the local network. Nodes on the other side of a router are not within reach of this broadcast.

The network must also be low latency. The `ElectionManager` relies on fixed timing assumptions to determine if other nodes are participating in the run. Very high latency may cause problems with this feature.

Implementation

The DRM is written in C++ using the C++ Standard Library and makes extensive use of Inter-Process Communication (IPC) mechanisms such as sockets, signals and pthreads. These features are part of the Posix Standard (`https://posix.opengroup.org`) and are available in all conforming operating systems. Library dependencies include `libz`, `libpthread`, and `libcrypto`, a part of the OpenSSL Project (`https://www.openssl.org/`).

The DRM is implemented with a small number of C++ classes: `DistributedRunManager`, `TicketManager`, `Ticket`, and `ElectionManager`. The implementation discussion is organized using these classes.

DistributedRunManager

A single `DistributedRunManager` object is created per application to control the distribution. It accepts input from the application which includes a network port number and a network key. These values distinguish a "run" and all processes in the run must agree on their values.

The network port number is the port which the DRM will use for network communications. On Linux systems it is a number between 0 and 65535. Ports 49152-65535 are available for private use and the DRM should use a port number in this range.

The network key is simply a string passed between processes as a handshake. If the handshake doesn't agree, communications are ended.

When the `DistributedRunManager` class is first instantiated it is neither a client nor a server. It creates an `ElectionManager` to determine which mode it will use going forward. The `ElectionManager` returns either the address of the server or informs the process that it is the server. It is here that the application is committed to run as a server or client until the run is finished.

Server initialization To communicate with clients, the server starts a dispatcher. This is a collection of threads used for communication with clients. It starts with the server creating a thread which listens for new connections on the network port. This thread

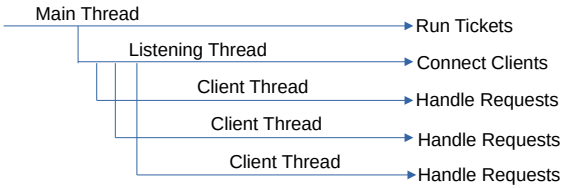


Figure 1: *The server process runs multiple threads to act as a dispatcher for client requests. The listening thread spawns a new thread for each client. The main thread goes on to run tickets.*

blocks while waiting for connections and persists for the life of the `DistributedRunManager`.

When a client connects, the listen thread creates a new thread specifically for that client's communications. A unique communication thread is created for every connected client. These threads *block* while waiting for client messages and persist only as long as the client is connected. Client requests for tickets are answered in these threads.

Meanwhile, the main thread continues on, running tickets in the server.

Client initialization The client mode of the DRM connects to the server, it exchanges the handshake, loads the parameters for the run, and then requests a ticket and *blocks* (waits) for the server to send some work.

Network Protocol The entire network protocol is comprised of three commands, each initiated by the client:

- Handshake
- Parameters
- Next Ticket

The client's first message to the server is its handshake string. Part of this string is the *network key*. This is a string determined by the application developer. Each application needs a unique string to prevent DRM-applications from colliding. If the server and client handshakes disagree, communications are ended.

Next, the clients request the input parameters for the run. Parameters are input to all the processes from the command line when they are started. However, significant time may pass between these various start times. Therefore, the server process provides the input parameters to each client guaranteeing consistency.

Finally, clients request work tickets. When the server responds with a new ticket, the client performs the work specified by that ticket and creates any necessary output files. This process repeats until all the tickets have been exhausted. Then the server returns an **EndOfTickets** message and the client exits.

Tickets

The *Ticket* is a data representation of a unit of work for the distribution, a single grain of its granularity. The two C++ classes `Ticket` and `TicketManager` are abstract base classes from which an application developer derives classes specific to their needs.

Ticket Class A `Ticket` variable is primarily a container for the information the application needs to run a single grain (i.e., an iteration) of the distribution. Often this is simply an integer.

Each instance has an identification number which corresponds to the iteration number of the distribution. The ticket ID is passed into the `Ticket` constructor and a unique ticket is made for each iteration.

The `Ticket` class also contains two methods for packaging its information for network transmission as a string: `TicketToBuffer()` and `BufferToTicket()`. Generally, these simply operate on the ticket ID. This is a *shallow* transfer of the ticket although true marshaling is possible.

TicketManager Class The `TicketManager` is designed as an interface between the `Ticket` class and the `DistributedRunManager` class. It is created once per application and each DRM must have a registered `TicketManager` in order to call `RunTickets()`.

This class has virtual methods for creating a new ticket, running the code for the ticket, and incrementing the ticket. Creating and incrementing the ticket are only performed on the server. Both server and clients, however, run the tickets.

Running Tickets The DRM takes control of the application when a call is made to the `DistributedRunManager.RunTickets()` method. It is called once and it retains control of the application until all tickets have been processed.

The `RunTickets()` method works the same in both server and client modes of the DRM. It simply requests the next ticket and runs it, until all the tickets have been handled. The request for the next ticket is always fulfilled by the server which tracks which tickets are ready to run. The `RunTickets()` method returns control to the application after all tickets have been completed.

The `TicketManager.HandleTicket()` method passes execution of the ticket to an application function using a callback function. This works identically for both server and client.

The `TicketManager` constructor requires a callback function and callback argument, e.g.:

```
    // assign callback function
    catlib::DRMCallbackFunc_t callback_func =
        run_the_ticket;

    // construct the ticketManager
    MyTicketManager ticketManager(callback_func,
        &myApp);
```

When the `HandleTicket()` method is called with a specific ticket argument, the `TicketManager` may unpack information in the ticket before passing it into the callback function, e.g.:

```
int MyTicketManager::HandleTicket(catlib::Ticket
    * ticket)
{
  MyTicket * my_ticket = (MyTicket *)ticket;

  int tckNumber = my_ticket->GetTicketNumber();

  CallbackFunc_(tckNumber, CallbackArg_);

  return(0);
}
```

The callback function in turn passes execution to the application. The application runs the iteration described by the ticket and generates output. When the callback returns, execution is passed back to the DRM. This repeats until the tickets are exhausted.

```
void run_the_ticket(int ticket, void * arg)
{
  MyApp * my_app = (MyApp *) arg;

  my_app->doCalculation(ticket);

  return;
}
```

DRM Fault Tolerance The DRM server records each ticket in a file as they are completed. In the event of a *server* crash, the entire run may be restarted by enabling the restart option. This option instructs the DRM to skip all tickets recorded in the restart file and restart the run where it left off.

Additionally, whenever the `RunTickets()` method gets to the last ticket in the server, the restart file is searched for missing tickets due to the possible premature exit of a client process from the distribution. Any missing tickets are (re-)run by the server to assure completion of the run.

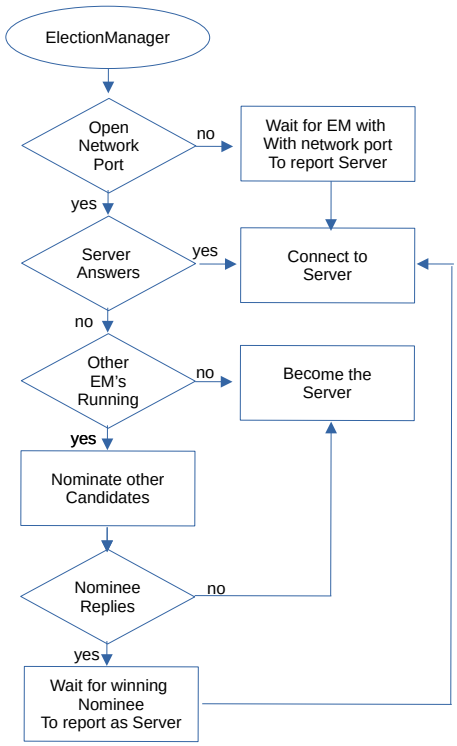


Figure 2: *The ElectionManager negotiates asynchronously with other Election Managers running on the host and/or network to determine which will become the server.*

ElectionManager

A DRM distribution must have one and only one server. If only a single DRM application is started, it will be its own server. When multiple are started, only one of them may be the server and the remaining must be clients.

The `ElectionManager` cooperates in a pool of other applications using the DRM to select a server for the distribution. It is not called directly by the application. Instead a single instance is created by the `DistributedRunManager` when it starts.

When the `ElectionManager` starts it attempts to open a network port on the local machine. Only one process may bind to a network port on a host. If multiple `ElectionManagers` are started *on the same host* only one of them (the first) will be able to open the network port. This is the *Network* instance.

Subsequent calls to attach to that port with the `bind()` system call return an `EADDRINUSE` error. Although these instances cannot open the network port they will still run, but they run as *Local* instances and have no access to the network. They are instead connected to the *Network* instance on their host.

Network ElectionManager If opening the network listening socket is successful (i.e., the *Network* instance), the `ElectionManager` broadcasts inquiry messages to the network repeatedly for `TMOUT_INQUIRY` seconds. If a server is available, it responds with its IP address. This information is passed back to the DRM which will then connect as a client to that address.

If no servers answer during `TMOUT_INQUIRY` seconds *and* no inquiries arrive from other processes also seeking a server, then this process becomes the server.

However, if no server answers after `TMOUT_INQUIRY` seconds, and there *are* other processes seeking the server, then an election is started. Those other processes seeking a server form a candidate list.

The candidate list is sorted by IP address. All IP addresses greater than "mine" are put into a nominee list and a nomination message is sent to each nominee. Meanwhile, every nomination message received is answered with a nomination accepted message.

After `TMOUT_ELECTION` seconds the list of hosts accepting their nomination is examined. If the list is empty, the process sets itself as server, i.e, it answers all pending and new broadcast messages seeking the server with its own address. If the list is not empty, an inquiry broadcast is repeated until the elected server reports back.

Local ElectionManager Processes without access to the network port (i.e., the *Local* instances) open a *unix* socket with which they can communicate with the *Network* process running on their host machine. When the *Network* process has determined the server's IP address, it informs all *Local* processes which may then connect to the network port of the server and act as clients there.

Post Election After a server is selected, its `ElectionManager` responds to new broadcasts on the election port with its IP address and all processes starting after the election will be directed to the same server.

Caveats The `ElectionManager` uses asynchronous communication on the network. It is impossible to plan for for every situation which may arise. Network or host configuration issues or extreme load conditions may cause the election to fail.

If problems persist, the `DistributedRunManager` provides constructors for specifically starting server or client objects and then the `ElectionManager` is removed from the process.

The election also adds latency to each process. As much as 4 seconds is required for an election. This is negligible for a typical DRM-application which may be running for hours or tens of hours.

Conclusion

Much effort has been devoted to distributed computing [4] with the advent of less expensive computers and more reliable network connections. However, applications must be modified to leverage the efficiencies of these distribution systems. The DRM addresses this problem.

Not all applications are candidates for distributed computing. However, for those problems which can benefit from distributed computation, the DRM offers a reusable tool which provides a single interface to the distribution problem.

The DRM interface hides the message passing, client-server selection process, client enrollment, and error handling of the distribution. It scales without modification from a single process to a High Performance Compute Cluster.

The DRM has been used for many years, for varied application types, and in many different distribution environments. It is a robust and flexible solution for transforming a serial application into a distributed application.

Acknowledgments

The DRM was first begun in the 2003-2004 time frame as a solution to increase throughput of Monte Carlo analyses, specifically for the NSECT program [7]. Its steady development over the 20-plus years since then was made possible by Duke University's Radiology Department.

Originally it ran on single hosts, was migrated to a Beowulf cluster, then to the Duke University's compute cluster and finally to the Duke Radiology Compute Cluster, a High Performance GPU based cluster. As the years passed, the research projects supported by the DRM were numerous and varied, branching into image processing, synthetic image generation, and general purpose distributed calculations.

The author is grateful for the support and patience of the many researchers he worked with during these decades. Without the support of Rai Labs Advanced Imaging (`https://rai.labs.duke.edu`) and the Center for Virtual Imaging Trials (`https://cvit.duke.edu`) this work would never have been completed.

References

[1] Nizar ALHafez and Ahmad Kurdi. *Parallel Paradigms in Modern HPC: A Comparative Analysis of MPI, OpenMP, and CUDA*. 2025. arXiv: 2506.

15454 [cs.DC]. URL: https://arxiv.org/abs/2506.15454.

[2] Brian P Harrawood et al. "Geant4 distributed computing for compact clusters". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 764 (2014), pp. 11–17.

[3] Sea Agostinelli et al. "Geant4—a simulation toolkit". In: *Nuclear instruments and methods in physics research section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 506.3 (2003), pp. 250–303.

[4] Md Firoj Ali and Rafiqul Zaman Khan. "Distributed computing: An overview". In: *International journal of advanced networking and applications* 7.1 (2015), p. 2630.

[5] Ramchandra Phawade, Tephilla Prince, and S Sheerazuddin. "Verification of Unbounded Client-Server Systems with Distinguishable Clients". In: *arXiv preprint arXiv:2602.23054* (2026).

[6] Donald J Becker et al. "BEOWULF: A parallel workstation for scientific computation". In: *Proceedings, international conference on parallel processing*. Vol. 95. 1995, pp. 11–14.

[7] Carey E Floyd Jr et al. "Neutron stimulated emission computed tomography of stable isotopes". In: *Medical Imaging 2004: Physics of Medical Imaging*. Vol. 5368. SPIE. 2004, pp. 248–254.